



INFORME TÉCNICO

PixelForge MathEngine 2D v1.0

Informe técnico y revisión bibliográfica

Universidad CENFOTEC · Escuela de Fundamentos
FUN-06 · Álgebra Lineal · Sección FCV0 · II Cuatrimestre 2026
Profesor: Steven Gabriel Sánchez Ramírez

Integrantes:

Alejandro Gabriel Medrano Ruiz
Antonio Mora Blotta
José Andrés Picado Corrales
Marcos Antonio Morales Céspedes

30 de julio de 2026

Tabla de contenido

1. Resumen ejecutivo	3
2. Objetivos, alcance y método	4
3. Revisión sistemática de literatura	5
4. Fundamentos de transformaciones y motores 2D	7
5. Descripción técnica y arquitectura	8
6. Algoritmos matemáticos	10
7. Resultados y casos de prueba	12
8. Limitaciones, mejoras y retroalimentación	14
9. Conclusiones	15
10. Referencias	16
11. Anexo de trazabilidad	17

Este informe integra la revisión bibliográfica y el informe técnico solicitados por la rúbrica. Los pseudocódigos, explicaciones detalladas y bitácoras individuales permanecen como anexos editables dentro de docs/.

1. Resumen ejecutivo

PixelForge MathEngine 2D v1.0 es un prototipo académico que representa figuras bidimensionales mediante coordenadas cartesianas, aplica transformaciones geométricas con matrices, conserva el historial de modificaciones y analiza los vectores asociados a los objetos. Su propósito es demostrar de forma visible y reproducible cómo el álgebra lineal sostiene operaciones habituales de un motor gráfico.

El sistema implementa las cuatro actividades de la consigna: representación matricial de objetos, traslación, rotación, escalamiento y reflexión, transformaciones consecutivas, análisis de espacios y subespacios, independencia, base, dimensión y redundancia, e historial de estados y matrices. La interfaz Tkinter presenta la figura original y la transformada, sus coordenadas, la matriz utilizada, el cálculo matricial, el análisis y el historial; Matplotlib produce la representación gráfica.

Las operaciones fundamentales se desarrollaron en el equipo. El cálculo de rango, base e independencia usa eliminación de Gauss y Gauss-Jordan propias. No se emplean `numpy.linalg.matrix_rank`, `numpy.linalg.inv` ni `numpy.linalg.eig`; NumPy se limita a representar arreglos, multiplicar matrices y apoyar comparaciones de prueba.

La validación realizada el 30 de julio de 2026 ejecutó 14 casos gestionados por unittest y 15 escenarios adicionales con aserciones directas. Todos finalizaron sin errores. El caso integrado rotó un cuadrado 30 grados, lo escaló por 1.5 y lo trasladó por (2, 1); el historial reconstruyó el mismo estado final mediante una única matriz homogénea compuesta.

El resultado es un núcleo funcional y explicable, adecuado para el alcance didáctico. Las principales oportunidades futuras son unificar todas las pruebas bajo unittest, persistir escenas e historiales, incorporar deshacer/rehacer y ampliar las reflexiones y transformaciones alrededor de pivotes configurables.

2. Objetivos, alcance y método

2.1 Objetivo general

Desarrollar un prototipo funcional de un motor matemático 2D que relacione modelos computacionales con resultados de álgebra lineal y permita verificar, visualizar y justificar cada operación implementada.

2.2 Objetivos específicos

- Representar cuadrados, triángulos, rectángulos y polígonos personalizados como matrices de puntos cartesianos.
- Aplicar traslación, rotación, escalamiento y reflexión sin modificar el objeto original.
- Encadenar transformaciones y componer sus matrices en coordenadas homogéneas.
- Calcular independencia lineal, base, dimensión y redundancia mediante algoritmos propios.
- Distinguir una colección finita de vértices del espacio que genera y analizar conjuntos definidos por $ax + by = c$.
- Registrar cada transformación con su matriz y los estados anterior y posterior.
- Presentar matrices, cálculos, coordenadas, interpretación geométrica y gráficos.
- Verificar el programa con casos reproducibles y documentar el uso de IA.

2.3 Alcance

El prototipo trabaja en $\mathbb{R}^2$ y transforma polígonos representados por una matriz $2 \times N$. Incluye una GUI de escritorio y un modo de demostración por consola para entornos sin pantalla. No pretende sustituir un motor de renderizado completo: no administra texturas, colisiones, animación temporal ni una jerarquía de escenas.

2.4 Método de desarrollo y validación

El trabajo se dividió en módulos con interfaces pequeñas. Cada integrante documentó pseudocódigo, fundamento matemático, consultas a IA y decisiones de validación. La integración se comprobó de dos maneras: aplicando cada transformación paso a paso y reconstruyendo el resultado desde la matriz compuesta. Las rotaciones se compararon con tolerancia numérica debido a la representación finita de seno y coseno.

3. Revisión sistemática de literatura

3.1 Preguntas de revisión

La revisión buscó responder tres preguntas: (1) ¿qué conceptos de espacios vectoriales justifican independencia, base, dimensión y rango?, (2) ¿cómo se representan y componen las transformaciones de objetos gráficos?, y (3) ¿por qué estas operaciones forman parte del núcleo matemático de un motor de videojuegos?

3.2 Estrategia y criterios

Se consultaron catálogos oficiales de editoriales académicas y el sitio del autor alojado por MIT. Se combinaron los términos “linear algebra”, “vector spaces”, “basis”, “dimension”, “computer graphics”, “transformations”, “game engine” y “homogeneous coordinates”. La búsqueda se cerró el 30 de julio de 2026.

Se incluyeron libros universitarios o profesionales con autoría identificable, edición y editorial verificables, relación directa con al menos una pregunta y tratamiento matemático o algorítmico. Se excluyeron entradas sin revisión editorial, tutoriales anónimos, páginas comerciales sin contenido técnico y fuentes que únicamente mostraran código sin justificación.

El corpus final reúne seis fuentes: dos textos de álgebra lineal y cuatro obras de gráficos o matemática para juegos. Esta combinación permite triangular la teoría de espacios vectoriales con su uso computacional.

3.3 Síntesis sobre espacios vectoriales

Strang (2023) organiza independencia, base y dimensión alrededor de los subespacios fundamentales y de los pivotes producidos por eliminación. Esta relación respalda la decisión de calcular la dimensión como rango y de conservar como base únicamente los vectores que aumentan ese rango. Lay, Lay y McDonald (2021) presentan la independencia como ausencia de una combinación lineal no trivial igual al vector cero y enfatizan que una base debe ser, simultáneamente, independiente y generadora.

En el prototipo los vértices se tratan como vectores de $\mathbb{R}^2$. Por ello, como máximo dos vectores independientes pueden formar una base del plano, aunque un polígono necesite más vértices para describir su contorno. La redundancia detectada es algebraica respecto al espacio generado; no implica que el punto deba eliminarse del dibujo.

3. Revisión sistemática de literatura — continuación

3.4 Subespacios y restricciones de posiciones

Un subconjunto es subespacio cuando contiene el vector cero y permanece cerrado bajo suma y producto por escalares. Para una restricción $ax + by = c$, el caso homogéneo $c = 0$ describe el núcleo de una transformación lineal y sí es un subespacio. Cuando $c \neq 0$, la recta es afín: puede ser útil para restringir el movimiento de un personaje, pero no contiene el origen y no es un subespacio.

Esta distinción corrige una posible lectura del ejemplo $x + y = 10$ del anexo. El sistema no afirma que toda restricción lineal sea un subespacio; muestra los cálculos y un contraejemplo. Si $p = (10, 0)$, entonces p pertenece, pero $2p = (20, 0)$ no satisface la misma ecuación.

3.5 Matrices y gráficos por computadora

Hughes et al. (2014) describen los gráficos como una combinación de modelado, interfaz y una secuencia de operaciones sobre representaciones geométricas. Esa separación se refleja en la arquitectura: Objeto2D modela la figura, Transformador opera sobre ella y interfaz.py presenta el resultado.

Akenine-Möller et al. (2018) sitúan las transformaciones como parte esencial del procesamiento gráfico en tiempo real. Aplicar una matriz a cada vértice permite transformar un objeto completo con una regla uniforme. Van Verth y Bishop (2016) y Dunn y Parberry (2012) conectan matrices, espacios de coordenadas y transformaciones con la implementación de juegos y motores interactivos.

Las coordenadas homogéneas permiten expresar la traslación como una matriz 3×3 . Al ampliar rotación, escalamiento y reflexión a ese mismo formato, todas las operaciones pueden componerse. Esta uniformidad simplifica el historial y la reconstrucción de escenas.

3.6 Conclusión de la revisión

La literatura converge en tres decisiones usadas por el proyecto: vectores columna para los puntos, pivotes para rango/base/dimensión y matrices homogéneas para componer transformaciones afines. La implementación no reproduce fórmulas de forma aislada: las integra en un flujo que conserva evidencia antes/después, expone el cálculo y permite validación numérica y visual.

4. Fundamentos de transformaciones y motores 2D

4.1 Representación de objetos

Para N vértices, la figura se almacena como:

$$P = \begin{bmatrix} x_1 & x_2 & \dots & x_N \\ y_1 & y_2 & \dots & y_N \end{bmatrix} \quad P \in \mathbb{R}^{(2 \times N)}$$

Cada columna es el vector posición de un vértice. Una transformación lineal $M \in \mathbb{R}^{(2 \times 2)}$ se aplica simultáneamente mediante $P' = M \cdot P$.

4.2 Matrices implementadas

$$\begin{aligned} \text{Rotación:} \quad R(\theta) &= \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \\ \text{Escalaiento:} \quad S &= \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \\ \text{Reflexión X:} \quad F_x &= \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \\ \text{Reflexión Y:} \quad F_y &= \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \\ \text{Reflexión } y=x: \quad F_d &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \\ \text{Traslación:} \quad T &= \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

La rotación conserva distancias y ángulos; el escalamiento multiplica cada componente por su factor; la reflexión cambia la orientación respecto a un eje; y la traslación suma el mismo desplazamiento a todos los puntos.

4.3 Composición

Con vectores columna, la matriz situada más cerca del punto actúa primero. Para rotar, luego escalar y finalmente trasladar:

$$\begin{aligned} p_{\text{final}} &= T \cdot S_h \cdot R_h \cdot p_{\text{inicial}} \\ C &= T \cdot S_h \cdot R_h \end{aligned}$$

El orden es significativo porque, en general, la multiplicación matricial no es conmutativa. Trasladar y después rotar también rota el vector de desplazamiento; rotar y después trasladar no.

5. Descripción técnica y arquitectura

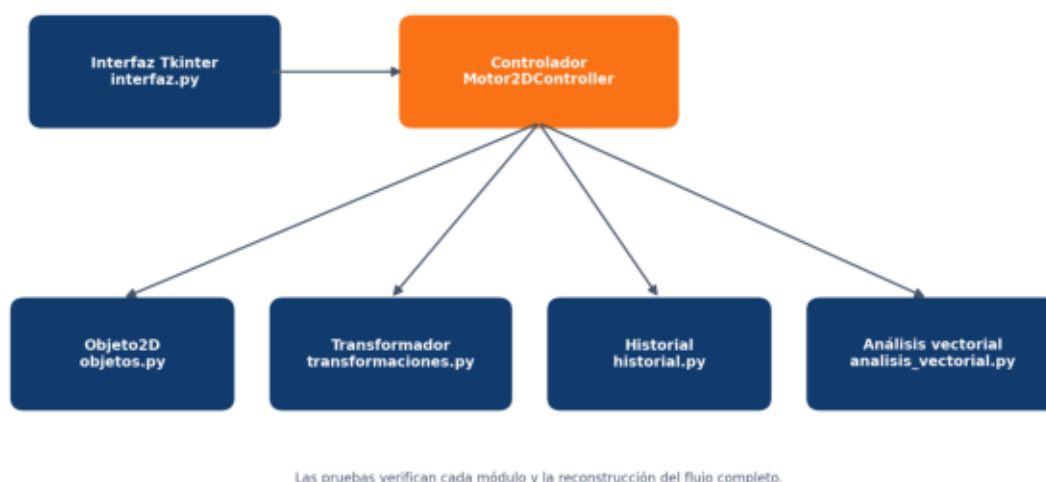


Figura 1. Arquitectura modular y dependencias principales.

5.1 Módulos

- objetos.py: valida puntos finitos, normaliza entradas $N \times 2$ o $2 \times N$, clona objetos y crea cuadrado, triángulo y rectángulo.
- transformaciones.py: construye las matrices y devuelve un nuevo objeto junto con la matriz usada.
- historial.py: guarda nombre, matriz y estados antes/después; compone matrices homogéneas y reconstruye el estado final.
- analisis_vectorial.py: implementa Gauss, Gauss-Jordan, base, dimensión, independencia, redundancia, cierre y análisis de $ax + by = c$.
- interfaz.py: ofrece un controlador independiente de Tkinter y una GUI que integra controles, resultados y gráficos.
- main.py: ejecuta el flujo principal por consola.

La separación evita duplicar algoritmos en la interfaz. Motor2DController coordina módulos ya probados; la ventana se limita a recibir datos, mostrar errores y presentar resultados.

5. Descripción técnica y arquitectura — continuación

5.2 Flujo principal

1. El usuario elige una figura predefinida o ingresa al menos tres pares (x,y).
2. Objeto2D convierte los puntos a una matriz $2 \times N$ y conserva el original.
3. El controlador solicita una transformación y sus parámetros.
4. Transformador valida números finitos, crea la matriz y calcula el nuevo estado.
5. HistorialTransformaciones registra copias independientes del antes y el después.
6. La interfaz actualiza coordenadas, cálculo $M \cdot P = P'$, matriz, historial y gráficos con escala común.
7. El analizador calcula base, dimensión, independencia y redundancia, o estudia una ecuación de posiciones como subespacio.

5.3 Decisiones de calidad

- Inmutabilidad práctica: las transformaciones crean un nuevo Objeto2D; no alteran el original.
- Copias en historial: cada registro conserva arreglos independientes para que cambios posteriores no reescriban el pasado.
- Tolerancia numérica: se usa $\text{EPSILON} = 1e-9$ para pivotes y allclose en validaciones, debido a redondeos de punto flotante.
- Errores explícitos: se rechazan coordenadas no finitas, figuras personalizadas con menos de tres puntos y ejes de reflexión no soportados.
- Algoritmos propios: el rango y la base no delegan el resultado a funciones directas de `numpy.linalg`.

5.4 Interfaz y presentación

La GUI organiza controles, dos gráficos y pestañas de resultados. Presenta la figura original, la transformada, la última matriz, el desarrollo matricial, base, dimensión, redundancia, interpretación y el historial completo. El modo --demo reproduce el caso oficial aun cuando no exista servidor gráfico.

6. Algoritmos matemáticos

6.1 Gauss-Jordan, rango e independencia

El algoritmo recorre columnas, busca un pivote no nulo, intercambia filas, normaliza el pivote y elimina su columna. El número de pivotes es el rango. Si el rango coincide con la cantidad de vectores de entrada, el conjunto es linealmente independiente.

```
rango ← 0
para cada columna:
    buscar pivote desde la fila rango
    si existe:
        intercambiar filas y normalizar el pivote
        eliminar la columna en las demás filas
    rango ← rango + 1
```

6.2 Base y redundancia

Los vectores se examinan en el orden recibido. Un vector se agrega a la base solo cuando incrementa el rango de los candidatos ya elegidos. Los índices no seleccionados son redundantes para generar el mismo espacio.

6.3 Cierre

El programa separa dos objetos matemáticos. La colección finita de vértices se comprueba buscando si cada suma y una muestra explícita de productos escalares pertenecen a esa misma colección. El espacio generado por los vértices, en cambio, siempre es cerrado y se describe mediante base y dimensión. Esta distinción se informa al usuario para evitar una conclusión excesiva a partir de una muestra finita.

6.4 Análisis de $ax + by = c$

Si $a = b = c = 0$, el conjunto es $\mathbb{R}^2$, con dimensión 2. Si $a = b = 0$ y $c \neq 0$, el conjunto es vacío. Si $c = 0$ y el vector normal no es nulo, la solución es una recta por el origen con base $[(-b, a)]$ y dimensión 1. Si $c \neq 0$, se construye un punto solución y su doble como contraejemplo de cierre.

6. Algoritmos matemáticos — historial y cálculo visible

6.5 Registro y reconstrucción

Cada transformación crea un registro con cuatro elementos: nombre, matriz, estado anterior y estado posterior. Para componer, las matrices 2×2 se amplían a 3×3 y cada operación nueva se multiplica por la izquierda:

```
C ← I3
para cada registro en orden:
    C ← matriz_homogénea(registro.matriz) · C
```

Aplicar C al objeto inicial debe coincidir con aplicar cada transformación por separado. Esta equivalencia es una invariante comprobada en las pruebas.

6.6 Desarrollo mostrado por la interfaz

Para la última operación la interfaz presenta M, los puntos de entrada, el producto y los puntos de salida. En la traslación se agrega una fila de unos y se explica la proyección de regreso a las dos primeras filas. El usuario puede consultar todas las matrices intermedias y la compuesta.

6.7 Complejidad

Aplicar una transformación a N vértices cuesta $O(N)$. La eliminación sobre una matriz de m vectores y n componentes tiene costo cúbico en el caso general, aunque para este motor $n=2$ y el tamaño es pequeño. La verificación exhaustiva de suma en la colección finita cuesta $O(m^3)$ con la búsqueda lineal implementada; es suficiente para escenas didácticas pequeñas.

Los pseudocódigos completos se encuentran en docs/pseudocodigos/ y las justificaciones extendidas en docs/explicaciones/.

7. Resultados y casos de prueba

7.1 Entorno y ejecución

La validación se ejecutó desde la raíz con Python 3:

```
python3 -m unittest discover -s tests -p "test_*.py" -v
Resultado: Ran 14 tests ... OK
```

Además de esos 14 casos, la importación de `test_analisis.py` y `test_transformaciones.py` ejecuta 15 escenarios con aserciones directas. En total se observaron 29 escenarios sin errores. Entre ellos se cubren:

- creación y clonación de objetos;
- traslación, rotación, escala y tres reflexiones;
- composición, historial y preservación del original;
- base, dimensión, independencia, redundancia y cierres;
- rectas homogéneas, rectas afines, R^2 y conjunto vacío;
- controlador de interfaz, validación de entradas y exportación PNG.

7.2 Caso completo 1: transformaciones consecutivas

Para el cuadrado $[(0,0), (2,0), (2,2), (0,2)]$ se aplicó rotación 30° , escalamiento uniforme 1.5 y traslación (2,1).

```
C = [[ 1.299038, -0.750000, 2.000000],
      [ 0.750000,  1.299038, 1.000000],
      [ 0.000000,  0.000000, 1.000000]]

P' = [(2.000000,1.000000), (4.598076,2.500000),
      (3.098076,5.098076), (0.500000,3.598076)]
```

La reconstrucción mediante C coincide con la secuencia paso a paso dentro de $1e-9$.

7. Resultados y casos de prueba — evidencia

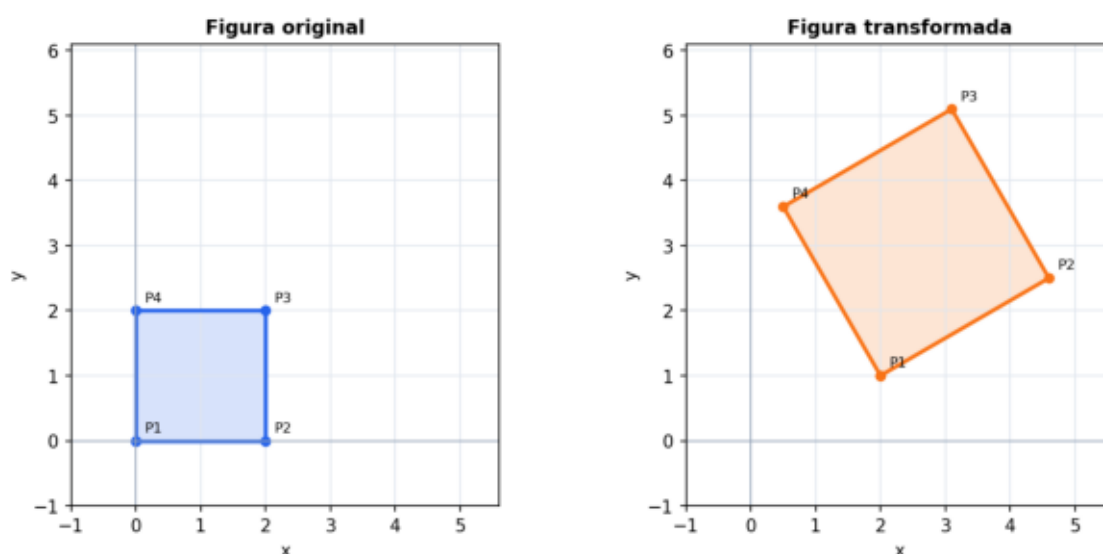


Figura 2. Resultado exportado por el modo de demostración: figura original y transformada con la misma escala.

La figura muestra que la forma del cuadrado se conserva bajo una rotación y escala uniformes, mientras la traslación cambia su posición. La comparación usa los mismos límites de ejes para no distorsionar la lectura geométrica.

7.3 Caso completo 2: subespacios

Para $x + y = 10$, el origen no pertenece. El punto $(10,0)$ satisface la ecuación, pero $(20,0) = 2(10,0)$ no; por ello no existe cierre bajo suma ni producto escalar y la recta no es subespacio.

Para $x + y = 0$, el origen sí pertenece y las propiedades de cierre se demuestran simbólicamente. Una base es $\{(-1,1)\}$ y la dimensión es 1. La comparación valida tanto un resultado negativo como uno positivo, con justificación matemática e interpretación geométrica.

7.4 Caso de redundancia

Con seis vectores $\{(1,0),(0,1),(2,0),(0,2),(1,1),(3,0)\}$, el algoritmo selecciona $\{(1,0),(0,1)\}$ como base, obtiene dimensión 2 y marca los índices 2, 3, 4 y 5 como redundantes para generar $\mathbb{R}^2$.

8. Limitaciones, mejoras y retroalimentación

8.1 Limitaciones conocidas

- El motor está limitado a R^2 ; por ello una base no puede contener tres direcciones independientes, aunque el anexo muestre un ejemplo genérico de dimensión 3.
- La comprobación de cierre escalar sobre vértices usa la muestra $[0, 1, -1, 2, -2, 0.5]$. Sirve para exhibir contraejemplos, pero no sustituye una demostración para todos los escalares reales.
- Las rotaciones y escalas se realizan respecto al origen; no existe aún un pivote arbitrario.
- El historial reconstruye y muestra estados, pero no expone botones de deshacer/rehacer ni persistencia en disco.
- Dos archivos de prueba conservan escenarios ejecutados al importar el módulo; funcionan, pero sería más uniforme convertirlos en clases unittest.

8.2 Mejoras propuestas

1. Migrar todos los escenarios a unittest y añadir medición de cobertura.
2. Incorporar deshacer/rehacer sobre los estados del historial.
3. Guardar y cargar figuras e historiales en JSON.
4. Añadir transformación alrededor del centroide o un pivote elegido.
5. Incluir reflexiones respecto a una recta general y cizallamiento.
6. Optimizar búsquedas de pertenencia para conjuntos grandes.
7. Automatizar la construcción del ZIP y la revisión de nombres antes de Moodle.

8.3 Retroalimentación del docente

No se encontró en el repositorio una devolución del docente correspondiente al primer avance. Por integridad académica no se inventaron observaciones. La matriz de cumplimiento y el seguimiento dejan este punto como pendiente de insumo externo. Si existe retroalimentación en Moodle o por correo, debe incorporarse aquí y reflejarse en las decisiones o correcciones realizadas antes de la entrega.

9. Conclusiones

El proyecto demuestra que una representación matricial compacta permite construir un núcleo coherente para objetos gráficos 2D. Al almacenar un vértice por columna, una misma multiplicación transforma todos los puntos y conserva una correspondencia directa entre la fórmula matemática, el código y el gráfico.

Las coordenadas homogéneas resuelven de forma uniforme la composición de transformaciones lineales con la traslación. El historial no es solo un registro de interfaz: permite reconstruir matemáticamente el estado final y verificar que el orden de las matrices coincide con el orden de operaciones.

El análisis vectorial aporta una segunda dimensión al prototipo. Gauss y Gauss-Jordan propios permiten explicar rango, base, dimensión e independencia sin ocultar el algoritmo en funciones de biblioteca. Asimismo, separar una colección finita de vértices de su espacio generado evita interpretar incorrectamente el cierre o la redundancia.

El estudio de $ax + by = c$ evidencia una conclusión especialmente relevante: una ecuación útil para restringir posiciones no necesariamente define un subespacio. El ejemplo $x+y=10$ es una recta afín y el sistema lo justifica con el origen y un contraejemplo de cierre.

La IA aceleró propuestas de estructura, pruebas, interfaz y documentación, pero cada aporte se sometió a decisiones del equipo, validación matemática y modificaciones explícitas. Esa trazabilidad, reunida en la bitácora grupal, mantiene a la herramienta en el papel de asistente indicado por la consigna.

En conjunto, PixelForge MathEngine 2D v1.0 cumple el alcance funcional y documental verificable. Quedan fuera únicamente el empaquetado final y la incorporación de cualquier retroalimentación docente que no está disponible en el repositorio.

10. Referencias

Akenine-Möller, T., Haines, E., Hoffman, N., Pesce, A., Iwanicki, M., & Hillaire, S. (2018). Real-time rendering (4.^a ed.). A K Peters/CRC Press. <https://www.realtimerendering.com/>

Dunn, F., & Parberry, I. (2012). 3D math primer for graphics and game development (2.^a ed.). A K Peters/CRC Press.

Hughes, J. F., van Dam, A., McGuire, M., Sklar, D. F., Foley, J. D., Feiner, S. K., & Akeley, K. (2014). Computer graphics: Principles and practice (3.^a ed.). Addison-Wesley Professional.

Lay, D. C., Lay, S. R., & McDonald, J. J. (2021). Linear algebra and its applications (6.^a ed.). Pearson.

Strang, G. (2023). Introduction to linear algebra (6.^a ed.). Wellesley-Cambridge Press. <https://math.mit.edu/~gs/linearalgebra/ila6/indexila6.html>

Van Verth, J. M., & Bishop, L. M. (2016). Essential mathematics for games and interactive applications (3.^a ed.). A K Peters/CRC Press.

Las referencias se normalizaron según APA 7. La revisión utiliza paráfrasis y síntesis propias; no reproduce fragmentos extensos de las fuentes.

11. Anexo de trazabilidad

Evidencia principal por actividad

- Actividad 1: src/objetos.py, pruebas de creación/clonación y pseudocódigos de José.
- Actividad 2: src/transformaciones.py, cálculo visible en src/interfaz.py, pruebas de las cuatro operaciones y Figura 2.
- Actividad 3: src/analisis_vectorial.py, pestañas de análisis y subespacio, pruebas de base, dimensión, redundancia y ecuaciones.
- Actividad 4: src/historial.py, flujo de src/main.py y reconstrucción en la prueba integrada.
- IA responsable: bitácoras individuales en docs/bitacoras/ y Bitacora_IA_Grupal.pdf.
- Pseudocódigo y justificación: docs/pseudocodigos/ y docs/explicaciones/.
- Ejecución y dependencias: README.md y requirements.txt.

Comandos reproducibles

```
python3 -m pip install -r requirements.txt
python3 -m unittest discover -s tests -p "test_*.py" -v
python3 src/interfaz.py
python3 src/interfaz.py --demo --grafico comparacion.png
python3 docs/generar_documentacion.py
```

La matriz detallada requisito por requisito se entrega en docs/MATRIZ_CUMPLIMIENTO.md. Esta indica también los dos asuntos que requieren acción externa o de entrega: la retroalimentación docente no disponible y la creación del archivo ZIP definitivo.